Haar Wavelet Image Watermarking in MATLAB: Interface, Recovery, and Evaluation

Abstract

This project examines a MATLAB application that embeds a prepared grayscale image watermark into another image using 2x2 Haar transforms. The GUI lets users load both images, choose watermark size and embedding strength, inspect each processing stage, and save results. After checking cover capacity, the program stores repeated watermark bits in detail coefficients and records decoding settings inside the image. Extraction reads those settings, reconstructs the grayscale watermark, and checks recovered bytes without requiring the original cover image. In the supplied example, the prepared 32 × 32 watermark was recovered exactly, with zero bit errors and a verified checksum. The same result reported 44.29 dB PSNR and 0.9966 block SSIM, describing cover quality for that particular configuration. The optional study tests image changes, but the available screenshot does not show complete labels for every attack result. This project therefore explains the implemented calculations and observed evidence while limiting conclusions to outcomes the MATLAB project actually shows.

1. Introduction

The project aims to place one image inside another while keeping the watermarked image useful and recoverable. Required functions include user interaction, two image inputs, strength control, embedding, extraction, display, and saving. The MATLAB interface brings these functions together through Create, Extract, Details, and Evaluate tabs shown during use. The cover image provides the visible picture, while the second image provides the information hidden in its pixels. The application prepares that second image as a smaller grayscale square before placing its bytes into Haar detail coefficients. This preparation matters because successful recovery must be judged against the stored square, not the larger input image. The user chooses strength and size, sees whether the cover has enough room, and then starts embedding. The interface also shows the original, prepared, watermarked, and recovered images so the processing stages remain understandable. Its Details tab reports image quality and recovery measures, while Evaluate tests selected changes to the watermarked image. The main question asks whether the program can recover its prepared watermark while making limited visible changes. A second question asks how strength, watermark size, and later image changes affect recovery and cover quality. The first question needs both matching recovered bytes and a measure of how much the cover image changed. The second needs several controlled settings because one attractive screenshot cannot describe performance under every image change. This project evaluates the MATLAB implementation and the supplied screenshots, without treating unseen experiments as completed results.

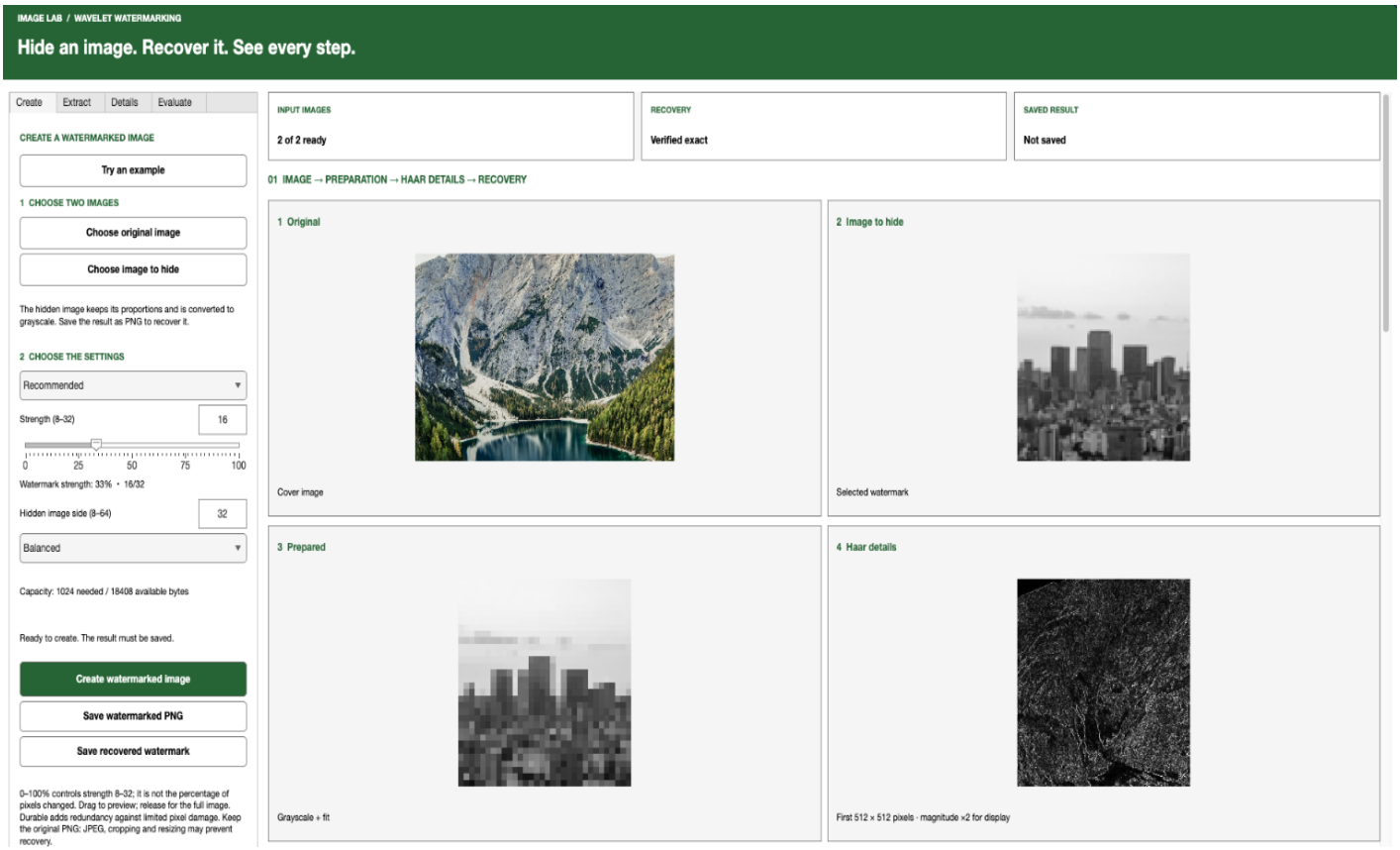


Figure 1: The Create tab shows both input images, strength controls, the prepared watermark, and a Haar detail preview.

2. Previous works

Hamidi and colleagues combined wavelet and cosine transforms while using SIFT features to address geometric changes (Hamidi et al.). Their method searches for stable image features after geometric distortion, then uses those features to support watermark recovery. That design targets a problem this project also tests: cropping and resizing can disturb the expected positions of embedded data. Their work also treats strength as a choice affecting visible quality and recovery, which supports testing several settings. Singh and colleagues studied blind extraction with adaptive scaling and block selection, emphasizing visual quality and robustness (Singh et al.). Blind extraction means the method does not need the original cover image to read a stored watermark from another image. The MATLAB project shares this practical goal because its extractor reads settings from a header embedded in the watermarked image. Singh and colleagues also select embedding locations and scaling values adaptively, illustrating ways to balance image quality against recovery. Li and colleagues added resampling detection and a template to address scaling and cutting attacks (Li et al.). Their approach first estimates the changed image geometry, which matters when a watermark reader must handle displaced blocks. The MATLAB extractor instead reads fixed Haar block positions, so its resizing and cropping results require direct measurement. These studies show why quality and recovery need separate measures, especially after compression or geometric image changes. The MATLAB project uses

smaller Haar blocks, repeated bits, and an embedded header instead of those published hybrid methods. Its header carries dimensions and decoding settings, while repeated bits and channel votes help recover individual payload bits. The project also checks recovered bytes and labels failures, which makes a plausible-looking preview less likely to be mistaken for success. Its contribution within this assignment is a complete interactive workflow with measurable recovery and explicit verification states. It does not claim to improve published attack resistance, since the three papers used different images, methods, and evaluation setups. The published results do not establish this program's robustness, so its own screenshots and study must support the conclusions.

3. System Design

The workflow begins with a cover image and a second image selected as the watermark through the Create tab. The program converts the selected watermark to grayscale, scales it without stretching, and pads it to the selected square size. Unused space in that square becomes white, so extraction represents the prepared watermark rather than the source pixels. The shown input watermark measures 96 × 96 pixels, while the prepared watermark measures 32 × 32 pixels. The interface accepts integer strength values from eight through thirty-two, with a slider offering matching percentages. For the shown strength of sixteen, the slider on the interface reads about thirty-three percent because (16 - 8) / (32 - 8) = 0.3333. The user selects Classic, Balanced, or Durable mode before the capacity check allows the embedding operation to proceed. After embedding, the program extracts the watermark immediately and displays the result beside the prepared watermark. The user can save a lossless PNG, reopen it for recovery, and save the recovered watermark separately.

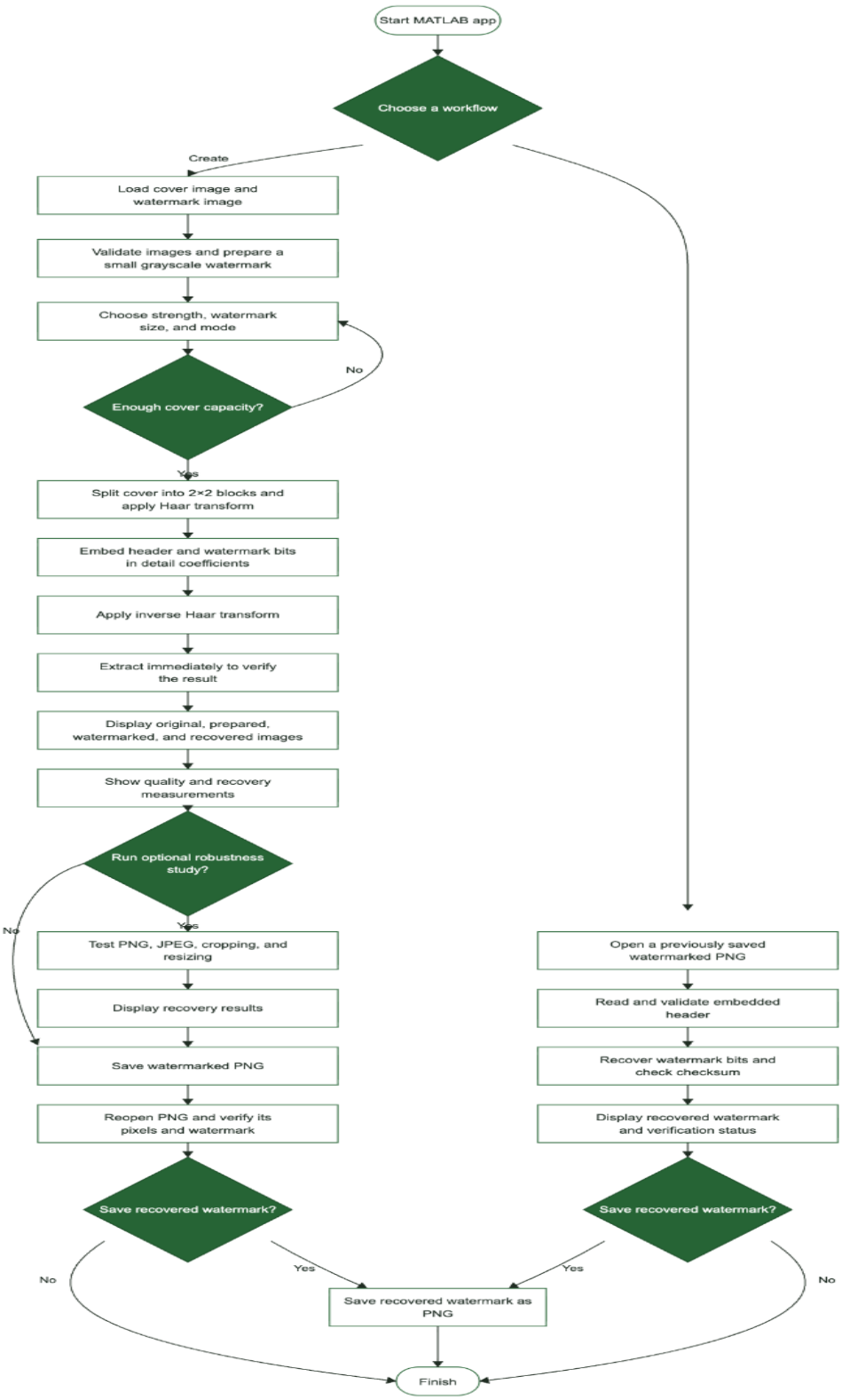


Figure 2: The workflow traces image preparation, Haar embedding, immediate verification, optional evaluation, PNG saving, and later extraction.

3.1 Haar transform and embedding

The MATLAB code divides each image channel into two-by-two blocks and applies the following Haar matrix.

$$H = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}, \qquad c = Hp, \qquad p = H^{\mathsf{T}}c$$

Here, p contains four pixel values, while c contains one average coefficient and three detail coefficients. The program writes watermark bits into the second and third coefficients, then applies the inverse transform to reconstruct pixels. For any coefficient c, watermark bit $b \in \{0,1\}$, and chosen strength s, the code applies the following quantization rule.

$$Q_s(c,b) = s\left[2\left\lfloor\frac{c/s - b}{2} + \frac{1}{2}\right\rfloor + b\right]$$

The bracketed value has the parity of b, making the encoded bit readable without the original cover. Larger strength values create wider quantization steps, although their actual effect on image quality must be measured. Balanced and Durable modes spread payload blocks across the cover, while Classic mode places them sequentially. Durable mode repeats each payload bit five times, whereas Classic and Balanced modes repeat each bit three times. Balanced and Durable modes also reduce strength in quiet blocks when the fourth Haar coefficient has magnitude below sixteen. For quiet blocks, the rule $s_q = \max(8, \lfloor 0.75s + 0.5 \rfloor)$ gives a local strength of twelve when the user selects sixteen.

$$s_q|_{s=16} = 12$$

3.2 Capacity calculation

Each two-by-two block offers two selected detail coefficients, giving two available embedding slots per channel. The code reserves 576 slots for its repeated 24-byte header before allocating any slots to the watermark. For cover height Hc, width Wc, and bit repetition count r, the payload capacity follows directly.

$$B = \left\lfloor\frac{H_c}{2}\right\rfloor\left\lfloor\frac{W_c}{2}\right\rfloor, \qquad C = \max\left(0, \left\lfloor\frac{2B - 576}{8r}\right\rfloor\right) \text{ bytes}$$

The shown cover measures 768 × 1152 pixels, so its usable two-by-two block count equals $B = \frac{768}{2} \cdot \frac{1152}{2} = 221{,}184$. With three payload copies in Balanced mode, the capacity formula gives $C = \left\lfloor\frac{2(221{,}184) - 576}{8(3)}\right\rfloor = 18{,}408$ bytes for this cover. The prepared 32 × 32 watermark needs $32 \cdot 32 = 1{,}024$ bytes, which is below the displayed capacity of 18,408 bytes. Repeating the 1,024-byte payload three times needs 24,576 slots, while the stored header adds another 576. Dividing by two shows that$B_{\text{used}} = \frac{24{,}576 + 576}{2} = 12{,}576$ cover blocks carry the header and payload, about $\frac{12{,}576}{221{,}184} \cdot 100\% \approx 5.69\%$ of available blocks. These values explain why the shown settings pass the capacity check without relying on a guessed image threshold.

3.3 Extraction and verification

Extraction reads the header first, checks its checksum, and then trusts the stored strength, size, and payload length. The decoder reads each marked coefficient and recovers its bit from the parity of the rounded strength multiple.

$$\hat{b} = \operatorname{mod}\left(\left\lfloor \frac{Q_s(c,b)}{s} + \frac{1}{2} \right\rfloor, 2\right)$$

Repeated copies vote on each bit, and color images also provide three channel copies for majority voting. The recovered bits form bytes, which the program reshapes into the grayscale dimensions recorded inside the valid header. A separate payload checksum then reports whether those recovered bytes match the original data written during embedding. Exact recovery compares the extracted image with the prepared 32 × 32 watermark, rather than the original 96 × 96 input. Later extraction from disk needs the unchanged watermarked PNG because its header stores the settings needed for decoding. Durable mode can show an approximate watermark after certain checksum failures, but the interface labels state that clearly.

4. Evaluation Method and Calculations

The application measures cover changes separately from recovery errors, so one can be successful without implying the other. For original cover samples (x_i) and watermarked samples (y_i), the code averages squared differences across all channels.

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(x_i - y_i)^2, \qquad \mathrm{PSNR} = 10\log_{10}\left(\frac{255^2}{\mathrm{MSE}}\right)$$

The shown 44.29 dB value implies an average squared pixel difference of about $\mathrm{MSE} = \frac{255^2}{10^{44.29/10}} \approx 2.42$ squared intensity units per sample. This conversion gives the displayed PSNR a concrete meaning without assigning an unsupported universal passing threshold. The code also computes luminance from $Y = 0.299R + 0.587G + 0.114B$ before measuring structure in nonoverlapping eight-by-eight blocks. Its block SSIM combines local means, variances, covariance, and constants $C_1 = (0.01 \cdot 255)^2$ and $C_2 = (0.03 \cdot 255)^2$ for stable division.$S_k = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \qquad \mathrm{BlockSSIM} = \frac{1}{K}\sum_{k=1}^{K} S_k$

The displayed SSIM is this project's block luminance variant, so comparisons should identify the exact calculation used. For prepared watermark W and recovered watermark $\widehat{W}$, bit error rate counts differing bits across every byte.

$$\mathrm{BER} = \frac{\sum_{i=1}^{P}\operatorname{popcount}\left(W_i \oplus \widehat{W}_i\right)}{8P}, \qquad \mathrm{NCC} = \frac{\sum_i W_i\,\widehat{W}_i}{\sqrt{\sum_i W_i^2 \sum_i \widehat{W}_i^2}}$$

Normalized correlation uses the pixel dot product, while the checksum and zero BER provide stronger evidence of exact recovery. The optional study tests three modes, three strengths, three watermark sizes, and eight specified image changes. The eight changes include unchanged PNG, three JPEG quality settings, two amounts of cropping, and two resize factors. For each change,

the recovery rate equals verified, pixel-identical recoveries divided by the number of tested configurations.

$$\text{. ExactRate} = \frac{\#\{\text{verified and pixel identical recoveries}\}}{\#\{\text{tested configurations}\}}$$

Settings that exceed capacity are skipped, and missing BER or NCC values mean extraction returned no comparable watermark.

5. Image Quality, Watermark Recovery, and Robustness Study

The supplied Details screenshot reports a 768 × 1152 cover, a 32 × 32 prepared watermark, and Balanced strength sixteen. The demonstration uses a mountain lake cover and a city skyline watermark, making their separate roles easy to identify. Figure 1 shows both selected images beside the prepared version, helping identify exactly which image is embedded. The 96 × 96 city image becomes a 32 × 32 grayscale square, so some original detail is necessarily lost. The interface displays capacity as 18,408 bytes available against 1,024 bytes needed for that prepared square. The displayed capacity follows the code's block count and repetition rule, not the cover's visible size alone. Its 33 percent slider label maps the selected strength within the allowed range and does not measure changed pixels. It labels recovery as verified exact, while showing 44.29 dB PSNR and a block SSIM value of 0.9966. Its BER reads 0.000 percent, and NCC reads 1.00000 for the prepared and recovered grayscale images. Because the same screen says the image was not saved, these values describe the current in-memory result. The zero BER means none of the 8,192 prepared watermark bits differed from the recovered watermark bits. NCC equal to one is consistent with that match, while the checksum confirms the recovered byte sequence. The 44.29 dB PSNR corresponds to about 2.42 squared intensity units of average error per cover sample. That error describes changes spread over the image and cannot identify the most visibly changed individual pixel. The block SSIM score near one indicates high measured structural similarity under this program's eight-by-eight calculation. Taken together, these values

support exact in-memory recovery with limited measured cover change for this chosen example.

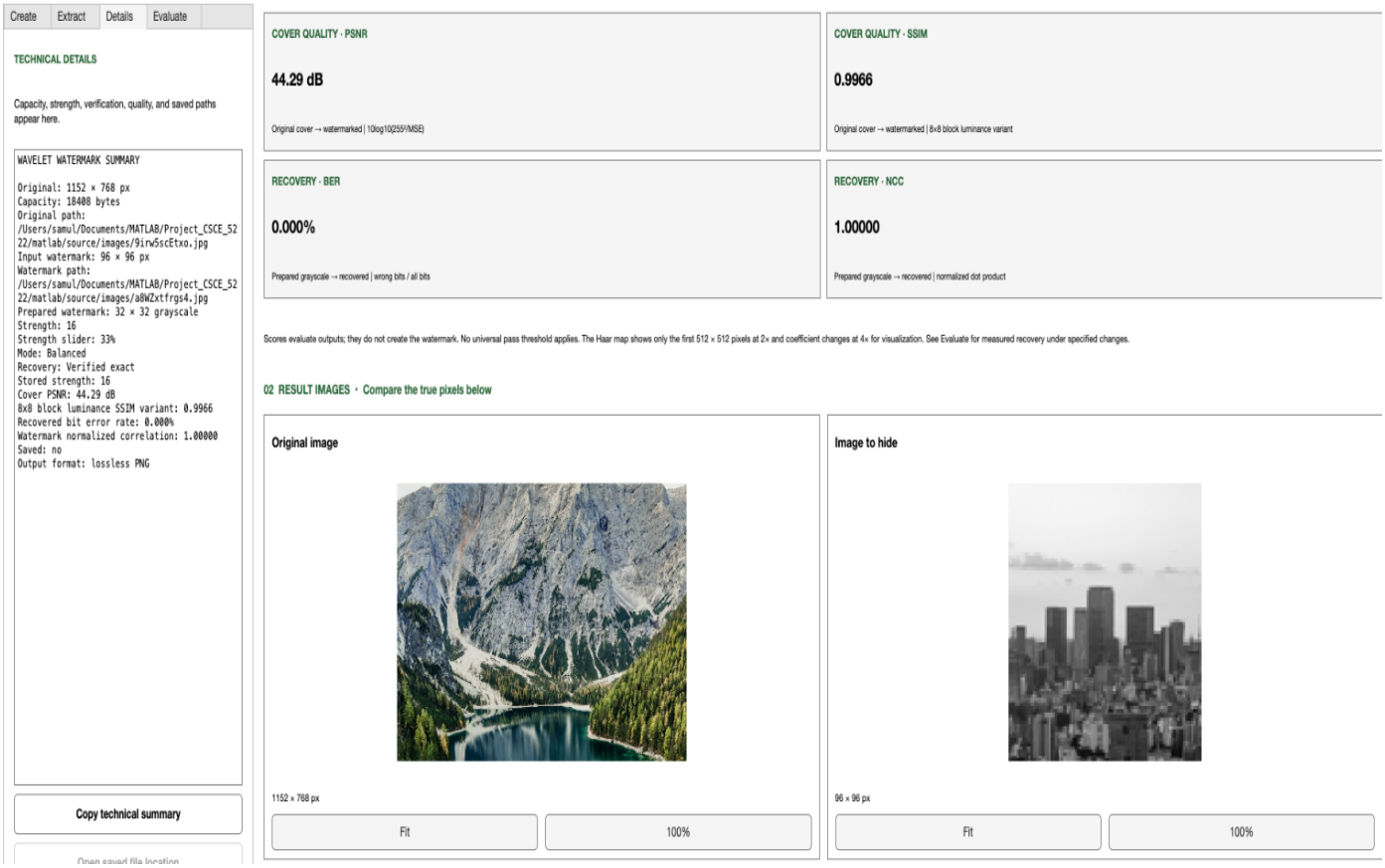


Figure 3: The Details tab shows verified recovery, cover quality scores, recovery scores, and the unsaved result state.

The Evaluate screenshot shows a separate study using a 512 × 342 working cover and nine capacity skips. The study reduces the cover before testing, which changes capacity and prevents direct comparison with the larger demonstration. Its code defines twenty-seven mode, strength, and watermark size settings before excluding those that exceed capacity. Nine skips leave eighteen usable settings, and eight changes per setting imply 144 evaluated change cases in that run. Its first summary row shows six tested cases and a 100 percent exact rate for unchanged PNG. Because the code lists Classic mode and unchanged PNG first, that row represents six fitting Classic settings after skipping. An exact rate of 100 percent here means every tested case in that row passed checksum verification and matched pixels. The same screenshot shows other zero percent rows, but their clipped labels prevent a reliable attack-specific claim. Those zero values alone cannot show whether a header failed, a payload failed, or a recoverable image differed. One visible detail row reports 48.0198 dB PSNR and 0.9977 block SSIM for its study configuration. Those study quality values describe a different working image and configuration, so they should not replace Figure 3 values. For any single setting, the study computes PSNR and block SSIM before applying its eight changes. Consequently, repeated quality values across several attack rows can describe one embedding result, rather than eight independent embeddings. The prepared and recovered previews appear closely matched, while the difference

preview is amplified four times for display. The comparison image is therefore a visual aid, not a direct measurement of fourfold embedding error.

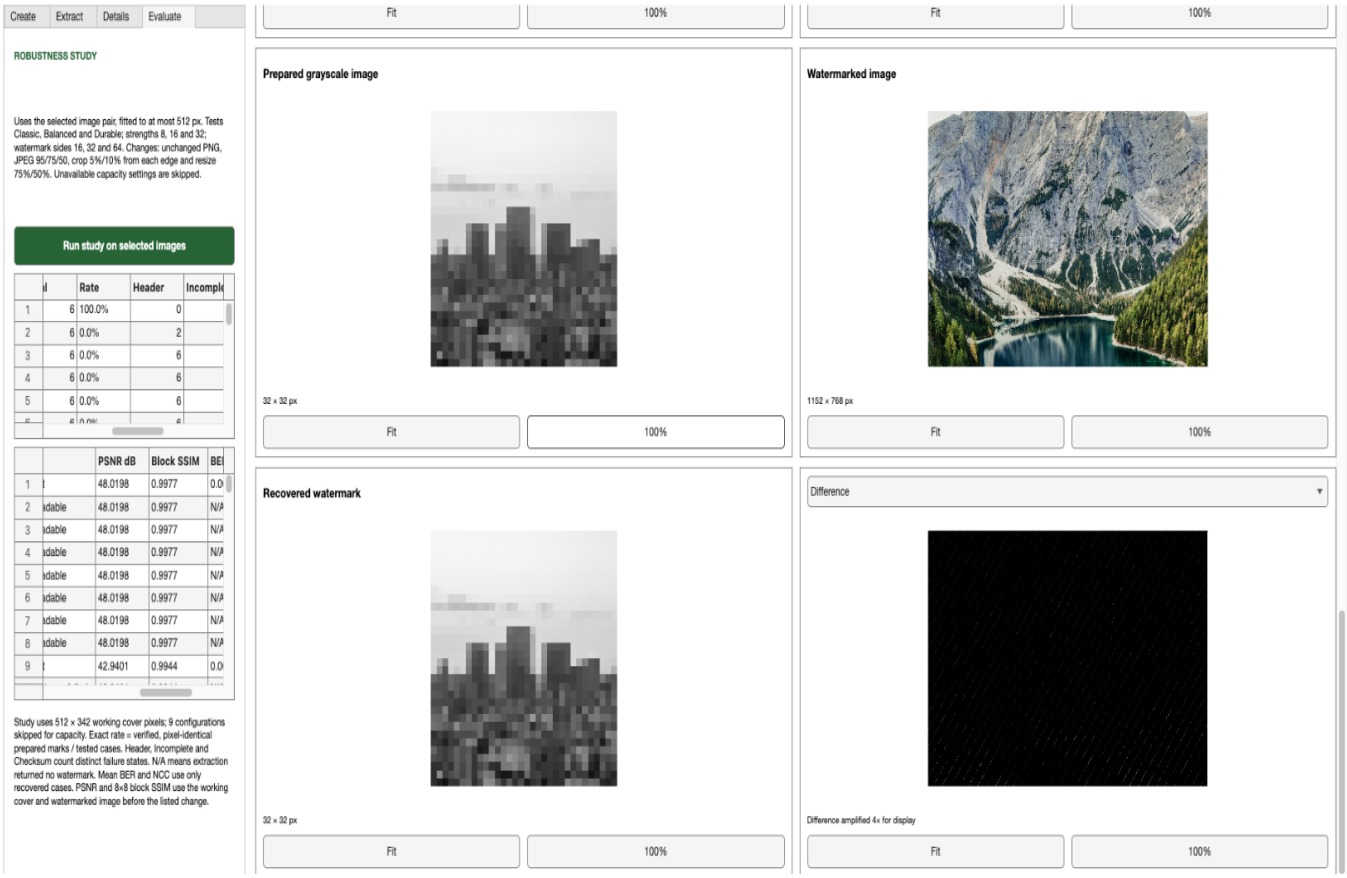


Figure 4: The Evaluate tab displays study rates, quality values, recovery previews, and capacity skips for tested settings.

The evaluator can return verified exact, payload checksum failure, unreadable header, or incomplete payload after an image change. Durable mode may return an approximate watermark when its payload checksum fails, and that output must not be called verified. JPEG compression may disturb coefficients, while cropping and resizing can move expected block positions and damage the stored header. When extraction returns no comparable watermark, the table reports unavailable BER and NCC rather than assigning zero error. This detail matters because average BER and NCC cover recovered cases only, while exact rates include every tested case. These are possible code outcomes, but the cropped screenshot shows only the values visible in its displayed columns.

6. Interpretation of Results and Project Limitations

The code and screenshots together demonstrate image loading, strength selection, embedding, extraction, display, and available PNG saving controls. Figure 1 makes the first assignment requirement visible through separate image buttons, setting controls, and image previews. The Create path takes two images, prepares the smaller one, checks capacity, and writes its bits into Haar details. The Extract path reads a previously watermarked image and uses

its stored header to reconstruct the grayscale watermark. These paths satisfy the requested embedding and extraction functions at code level, while the shown run demonstrates immediate recovery. The observed exact recovery answers the main project question for the shown in-memory image and prepared watermark. It shows that the 32 × 32 prepared square survived one embedding and extraction cycle without any bit changes. It does not mean the original 96 × 96 city image was restored, because preparation reduced its dimensions first. The capacity derivation also explains why the selected watermark fits, connecting the interface message to actual coefficient slots. The 1,024-byte payload occupies only part of the 18,408-byte calculated capacity for the displayed Balanced setting. That margin allows the example to proceed, but a larger square or Durable repetition can exhaust a smaller cover. The 44.29 dB PSNR and 0.9966 block SSIM describe modest measured change for this single shown configuration. They do not prove every image or strength will produce the same quality, because content and settings change coefficients. The strength slider controls quantization spacing, so stronger settings can increase coefficient changes while improving bit separation. That expected tradeoff remains a testable expectation here, since the supplied screenshots do not show matched strength comparisons. The study is designed to provide those comparisons across strengths eight, sixteen, and thirty-two for each fitting size. Its nine capacity skips should remain visible in any analysis because omitted settings narrow the tested combinations. Hamidi and colleagues used additional transforms and feature matching for geometric attacks, unlike this simpler MATLAB design (Hamidi et al.). Li and colleagues added explicit scaling and cutting correction, which this project's extraction code does not include (Li et al.). For that reason, its optional study should report attack results separately rather than suggesting general geometric robustness.

The visible unchanged PNG row supports lossless persistence in the study, where the program writes and rereads test images. The main screenshot still says unsaved, so it cannot prove the user completed the separate save and reopen path. The unreadable and checksum failure states matter because a visible recovered shape alone cannot prove valid embedded bytes. Likewise, correlation near one can coexist with some wrong bytes, while zero BER checks every bit against the prepared watermark. The checksum detects changed data, but it cannot establish ownership or protect the watermark against deliberate replacement. Its role is internal payload verification, and this implementation makes no claim of secure authentication. The screenshots do not show a completed save and reopen sequence, although the MATLAB code implements both operations. Figure 4 also clips several attack names, preventing a fair explanation of which changes produced the displayed failures. Complete exported study rows would support attack-specific rates, separate failure counts, and careful comparisons between modes. Future testing should capture a saved PNG being reopened and should retain complete study columns for each named attack. It should also repeat tests with several covers and watermark images because image texture affects the available detail coefficients.

7. Conclusion

The MATLAB project meets the requested design through a GUI that embeds and extracts an image watermark using Haar details. Its shown run recovers the prepared grayscale watermark exactly while reporting separate cover quality and recovery measures. The reported zero BER and verified checksum show exact stored bytes, while PSNR and block SSIM describe cover changes. The code's header, repeated bits, capacity check, and checksum explain how it produces and verifies this result. Its capacity calculation shows that the demonstrated 1,024-byte watermark fits within 18,408 available bytes at the selected settings. The interface also provides image previews and saving controls, connecting the required calculations to actions a user can follow. The available study adds a way to examine changed images, but cropped screenshot columns limit conclusions about individual attacks. The observed results establish one successful in-memory example and a visible unchanged PNG study row with exact recovery. Further evidence needs a completed user save-and-reopen sequence, plus fully labeled results for every tested change. The clearest project claim is exact recovery for the shown unchanged result, supported by zero BER and verified bytes.

Works Cited

Hamidi, Mohamed, et al. “A Hybrid Robust Image Watermarking Method Based on DWT-DCT and SIFT for Copyright Protection.” *Journal of Imaging*, vol. 7, no. 10, 2021, article 218, https://doi.org/10.3390/jimaging7100218.

Li, Hao-Lai, et al. “Resampling-Detection-Network-Based Robust Image Watermarking against Scaling and Cutting.” *Sensors*, vol. 23, no. 19, 2023, article 8195, https://doi.org/10.3390/s23198195.

sazzeb. “Project_CSCE_5222.” *GitHub*, https://github.com/sazzeb/Hough-Transform/tree/main/Project_CSCE_5222. Accessed 8 Nov. 2026.

Singh, Priyanka, et al. “Blind and Secured Adaptive Digital Image Watermarking Approach for High Imperceptibility and Robustness.” *Entropy*, vol. 23, no. 12, 2021, article 1650, https://doi.org/10.3390/e23121650.